

National Taiwan Ocean University
ICPC Codebook

yokAndHisBuddies

James Hong (yok939)

September 30, 2026

Contents

- 1. Misc 3
 - 1.1. Tools 3
 - 1.1.1. Macros 3
 - 1.1.2. <random> 3
 - 1.1.3. Floating Point Binary Search 3
 - 1.2. Algorithms 3
 - 1.2.1. Bit Tricks 3
 - 1.2.2. Alien Tricks 3
 - 1.2.3. Hilbert Curve 3
 - 1.2.4. Infinite Grid Knight Distance 3
 - 1.2.5. Longest Increasing Subsequence 3
 - 1.2.6. Mo's Algorithm on Tree 3
- 2. Data Structure 4
 - 2.1. 2DBIT.cpp 4
- 3. Demo 4
 - 3.1. Demo code 4

1. Misc

1.1. Tools

1.1.1. Macros

```
1 #define _GLIBCXX_DEBUG 1
2 #pragma GCC optimize("O3", "unroll-loops")
3 #pragma GCC optimize("fast-math")
4
```

1.1.2. <random>

```
1 #include <random>
2 #ifdef __unix__
3 std::random_device rd;
4 std::mt19937_64 RNG(rd());
5 #else
6 const auto SEED = chrono::high_resolution_clock::now()
7     .time_since_epoch()
8     .count();
9 std::mt19937_64 RNG(SEED);
10 #endif
11
```

1.1.3. Floating Point Binary Search

```
1 union di
2 {
3     double d;
4     unsigned long long i;
5 };
6 bool check(double);
7 double binary_search(double L, double R, int eps)
8 {
9     di lf = {L}, ri = {R}, mid;
10    while(ri.i - lf.i > 1ll << (52 - eps)) {
11        mid.i = (lf.i + ri.i) >> 1;
12        if(check(mid.d)) ri = mid;
13        else lf = mid;
14    }
15    return lf.d;
16 }
17
```

1.2. Algorithms

1.2.1. Bit Tricks

```
1 // next permutation of x as bit sequence.
2 ull next_bits_permutation(ull x)
3 {
4     ull c = __builtin_ctzll(x), r = x + (1ULL << c);
5     return (r ^ x) >> (c + 2) | r;
6 }
7 // iterate over all (proper) subsets of bitsets s.
8 void subsets(ull s)
9 {
10    for(ull x = s; x;)
11    {
12        --x &= s;
13        /* something */
14    }
15 }
16
```

1.2.2. Alien Tricks

```
1 // min dp[i] value and its i (smallest one)
2 pll get_dp(int cost);
3 ll aliens(int k, int lf, int ri)
4 {
5     while(lf != ri)
6     {
7         int mid = (lf + ri) / 2;
8         auto [f, s] = get_dp(mid);
9         if(s == k) return f - mid + k;
10        if(s < k) ri = mid;
11        else lf = mid + 1;
12    }
13    return get_dp(lf).F - lf + k;
14 }
15
```

1.2.3. Hilbert Curve

```
1 ll hilbert(ll n, int x, int y)
2 {
3     ll res = 0;
4     for(ll s = n; s /= 2;)
5     {
6         int rx = !(x & s), ry = !(y & s);
7         res += s * s + ((3 + rx) * ry);
8         if(ry == 0)
9         {
10            if(rx == 1) x = s - 1 - x, y = s - 1 - y;
11            swap(x, y);
12        }
13    }
14    return res;
15 }
16
```

1.2.4. Infinite Grid Knight Distance

```
1 ll hilbert(ll n, int x, int y)
2 {
3     ll res = 0;
4     for(ll s = n; s /= 2;)
5     {
6         int rx = !(x & s), ry = !(y & s);
7         res += s * s + ((3 + rx) * ry);
8         if(ry == 0)
9         {
10            if(rx == 1) x = s - 1 - x, y = s - 1 - y;
11            swap(x, y);
12        }
13    }
14    return res;
15 }
16
```

1.2.5. Longest Increasing Subsequence

```
1 template <class I> vector<int> lis(const vector<I> &S)
2 {
3     if(S.empty()) return {};
4     vector<int> prev(sz(S));
5     typedef pair<I, int> p;
6     vector<p> res;
7     for(int i = 0; i < sz(S); i++)
8     {
9         auto it = lower_bound(all(res), p{S[i], 0});
10        if(it == res.end()) res.emplace_back(), it =
11        res.end() - 1;
12        *it = {S[i], i};
13        prev[i] = (it == res.begin() ? 0 : (it - 1) -
14        >second);
15    }
16    int L = sz(res), cur = res.back().S;
17    vector<int> ans(L);
18    while(L--) ans[L] = cur, cur = prev[cur];
19    return ans;
20 }
```

1.2.6. Mo's Algorithm on Tree

```
1 void MoAlgoOnTree()
2 {
3     dfs(0, -1);
4     vector<int> euler(tk);
5     for(int i = 0; i < n; i++)
6     {
7         euler[tin[i]] = i;
8         euler[tout[i]] = i;
9     }
10    vector<int> lf(q), ri(q), qr(q), sp(q, -1);
11    for(int i = 0; i < q; i++)
12    {
13        if(tin[u[i]] > tin[v[i]]) swap(u[i], v[i]);
14        int z = LCA(u[i], v[i]);
15        sp[i] = z[i];
16        if(z == u) lf[i] = tin[u[i]], ri[i] = tin[v[i]];
17        else lf[i] = tout[u[i]], ri[i] = tin[v[i]];
18        qr[i] = i;
19    }
20    sort(all(qr), [&](int i, int j)
```

```

21     {
22         if(l[i] / kB == l[j] / kB) return ri[i] < ri[j];
23         return lf[i] / kB < lf[j] / kB;
24     });
25     vector<bool> used(n);
26     // Add(v): add/remove v to/from the path based on
used[v].
27     for(int i = 0; tl = 0, tr = -1; i < q; i++)
28     {
29         while(tl < lf[qr[i]]) Add(euler[tl++]);
30         while(tl > lf[qr[i]]) Add(euler[--tl]);
31         while(tr > ri[qr[i]]) Add(euler[tr--]);
32         while(tr < ri[qr[i]]) Add(euler[++tr]);
33         // add/remove LCA(u, v) if needed
34     }
35 }
36

```

2. Data Structure

2.1. 2DBIT.cpp

```

1  #include <bits/stdc++.h>
2  using ll = long long;
3  #define pii pair<int, int>
4  #define pll pair<ll, ll>
5  #define N 1005
6  #define F first
7  #define S second
8  #define lb(x) (x & -x)
9  using namespace std;
10 int bit[N][N];
11 void upd(int i, int j, int v)
12 {
13     for(; j < N; j += lb(j))
14         for(int k = i; k < N; k += lb(k)) bit[k][j] += v;
15 }
16 int qry2(int i, int j)
17 {
18     int ans = 0;
19     for(; j; j += lb(j))
20         for(int k = i; k; k += lb(k)) ans += bit[k][j];
21     return ans;
22 }
23 int qry(int y1, int x1, int y2, int x2)
24 {
25     return qry2(y2, x2) - qry2(y2, x1 - 1) - qry2(y1 - 1,
x2) + qry2(y1 - 1, x1 - 1);
26 }
27 int main()
28 {
29     // REMEMBER TO DO MEMSET
30     memset(bit, 0, sizeof(bit));
31     int n, q, i = 1, j, y, x;
32 }
33

```

3. Demo

3.1. Demo code

```

1  #include <iostream>
2
3  int main()
4  {
5      int n;
6      std::cin >> n;
7      for(int i = 0; i < n; i++)
8      {
9          std::cout << "Hello NTOU!\n";
10     }
11 }
12

```